AWS EC2 Auto Scaling Assignment

Static Web Server, Auto Scaling Group and Load Testing

Task 1 – Launch EC2 and Host a Static HTML Page

Steps:
1. Sign in to the AWS Management Console and open EC2.
2. Click **Launch instance** and select Amazon Linux 2023 (or the image provided by your sandbox).
3. Select a Free Tier eligible instance type, such as t2.micro/t3.micro where available.
4. Create/select a key pair and allow SSH (port 22) and HTTP (port 80) in the security group.
5. Launch the instance and wait until its status is Running.
6. Connect to the instance using SSH.
7. Install Apache HTTP Server and start it:

```
sudo dnf install -y httpd
sudo systemctl enable --now httpd
```

8. Create a simple page:

```
echo '<h1>Hello from AWS EC2</h1>' | sudo tee /var/www/html/index.html
```

9. Copy the EC2 Public IPv4 address and open **http://PUBLIC-IP** in a browser.

Public URL: http://<YOUR-EC2-PUBLIC-IP>
Evidence: Add a screenshot showing the EC2 instance and the HTML page in the browser.

Task 2 – Create an Auto Scaling Group

Steps:
1. Open EC2 → **Launch Templates** and create a launch template for the web server.
2. Use the same AMI, instance type, key pair, security group and web-server User Data/script used for Task 1.
3. Open EC2 → **Auto Scaling Groups** → Create Auto Scaling group.
4. Select the launch template.
5. Choose the required VPC and subnets.
6. Set **Desired capacity = 1, Minimum = 1, Maximum = 3**.
7. Create a scaling policy using target tracking or a suitable CloudWatch alarm so that CPU utilization is maintained around **60%**. If the assignment specifically requires 'above 60% for 5 minutes', configure a CloudWatch alarm with CPUUtilization > 60% and evaluation period of 5 minutes, then use it for scale-out.
8. Create the Auto Scaling Group and verify that one EC2 instance is running.

Setting	Required value
Minimum capacity	1
Desired capacity	1
Maximum capacity	3
Scale-out condition	CPU > 60% for 5 minutes

Task 3 – Simulate Increased Traffic and Observe Scaling

Steps:
1. Use a permitted load-testing service such as Loader.io or k6. Only test infrastructure you own or are

authorized to test.
2. Configure the test with the public endpoint of your AWS web server/load balancer.
3. Start with a small request rate and increase it gradually.
4. Open CloudWatch → EC2 → CPUUtilization and monitor the instances.
5. When CPU remains above the configured threshold for the required period, check EC2 → Auto Scaling Groups → Activity.
6. Verify that the desired/running capacity increases from 1 toward 2 or 3 instances.

Evidence: Add screenshots of the load test, CloudWatch CPU graph, and Auto Scaling Activity showing a new instance launch.

Task 4 – Reduce Load and Verify Scale-In

Steps:
1. Stop or reduce the load-testing traffic.
2. Continue monitoring CloudWatch CPUUtilization.
3. When the scale-in condition is satisfied according to your configured policy, AWS terminates an extra instance.
4. Open Auto Scaling Groups → Activity and verify the termination/scale-in event.
5. Confirm that the group returns to its minimum capacity of **1 instance**.
6. Take a screenshot of the Activity history showing the scale-in event and the final capacity.

Evidence: Insert screenshot here: *Auto Scaling Activity History – scale in to 1 instance*.

Task 5 – Real-World Auto Scaling Scenarios

1. Zomato – Lunch/Dinner Rush

During lunch and dinner hours, many users may simultaneously search restaurants, view menus and place orders. Auto-scaling can add more application servers when demand increases and reduce them later when traffic becomes normal. This helps maintain performance without permanently running the maximum number of servers.

2. Flipkart – Big Sale Events

During events such as major online sales, the number of visitors and requests can increase sharply. Auto-scaling can automatically increase compute capacity to handle extra traffic and reduce capacity after the sale. This makes infrastructure more flexible for short-term traffic spikes.

3. BookMyShow – Movie Release / Ticket Opening

When tickets for a popular movie or event open, a large number of users may try to access the booking system at nearly the same time. Auto-scaling can provide additional application capacity during the spike and scale down after demand decreases. This can help the service handle changing traffic efficiently.

Conclusion

AWS Auto Scaling helps maintain application availability and performance by adjusting the number of EC2 instances according to demand. In this assignment, the Auto Scaling Group is configured for 1–3 instances, with CPU-based scaling used to respond to increased workload.

Important: Replace the placeholder public URL and add your actual AWS screenshots before submitting. The PDF provides the procedure and answer format; it does not claim that the AWS resources were actually created.